MMoH: Efficient Training of Multimodal Large Language Models on Heterogeneous Clusters

Jinshan Fan, Zhiquan Lai*, Weijie Liu, Shengwei Li, Wei Wang, Yanqi Hao, Dongsheng Li
National Key Laboratory of Parallel and Distributed Computing(PDL)
College of Computer Science and Technology, National University of Defense Technology
Changsha, China
{fanjinshan,zqlai,liuweijie,swli,wwking,yqhao,dsli}@nudt.edu.cn

Abstract—Multimodal large language models (MLLMs) achieve strong performance across diverse AI applications but remain costly to train. With advancing hardware, heterogeneous clusters are increasingly used for MLLM training. However, existing methods often ignore MLLMs' hybrid-module structure and varying parameter states, leading to suboptimal performance. We propose MMoH, an efficient training approach for MLLMs on heterogeneous clusters. MMoH consists of a parallel strategy planner and an early-termination scheduler. The planner jointly considers model structure, device heterogeneity, and parameter states to select optimal parallel strategies. It first matches module requirements with device capabilities to identify feasible topologies. Then, using a hybrid-granularity abstraction, it divides the model into chunks and assigns pipeline stages accordingly. The scheduler further reduces overhead by eliminating redundant computation and communication. Experiments show that MMoH accelerates MLLM training on heterogeneous clusters by up to 1.77× over state-of-the-art methods.

Index Terms—heterogeneous clusters, multimodal large language models, parallel strategy generation

I. Introduction

Large language models (LLMs) [? ?] have achieved significant success in many areas, but their single-modality nature limits their ability to understand multimodal information. Multimodal large language models (MLLMs) [? ? ?] extend LLMs to learn from and generate content across multiple modalities, improving performance on tasks such as visual question answering [?], cross-modal translation [?], and integrated content understanding [?]. The substantial parameters of MLLMs necessitate parallel training with multiple devices. Pipeline parallelism (PP) [? ?], partitioning model layers across devices and orchestrating the training process in a pipeline manner, is widely adopted.

Driven by hardware diversification, distributed model training on heterogeneous clusters has emerged as a practical and cost-efficient solution. This trend is supported by superior price-performance in mixed-device deployments [? ?], reduced queuing delays for heterogeneous allocations on cloud platforms [?], and the widespread reliance on consumer-grade GPUs in academic research [?]. Unfortunately, the distinct computational performance and memory capacity of training devices in heterogeneous clusters pose significant challenges in effective parallel training. Recent efforts [? ? ?] have explored optimizing PP strategies to enhance the LLM training on heterogeneous clusters. However, these approaches are poorly suited for training MLLMs on heterogeneous clusters due to the following two characteristics of MLLMs.

Non-uniform model structure of MLLMs. Unlike LLMs composed of identical transformer layers [?], MLLMs typically comprise three core modules with distinct structures as demonstrated in Fig. ??: a modality-specific encoder, a task-oriented projector, and an LLM backbone. As shown in Fig. ??, LLMs show consistent computational cost and memory usage per layer. In contrast, MLLMs vary greatly in these aspects depending on the module. Their hybrid architecture complicates parallel training strategy design on heterogeneous clusters. Using existing heterogeneous training methods [? ?] for MLLMs leads to major inefficiencies. Current approaches perform up to 13.4% worse than the optimal parallelization scheme (Fig. ??).

Varying parameter training states of MLLMs. The frozen [?] technique is employed in the pre-training process of MLLMs, which bridges the modality gap, preserves valuable pretrained knowledge, and prevents overfitting. This technique results in a mixture of parameter training states—that is, the coexistence of frozen and active parameters. The heterogeneity between frozen and active parameters in their computational overhead and memory consumption (Fig. ??) means that the optimal parallel strategy is determined by the current mixture of parameter training states. Consequently, existing strategies are suboptimal, exhibiting performance gaps of up to 21.2% under common freezing scenarios (Fig. ??). Finally, the continual evolution of parameter states necessitates the dynamic adjustment of the parallel strategy across training phases. Unfortunately, existing approaches assume all parameters are always active during the overall training process, resulting in significant training inefficiencies.

To fill this gap, we propose MMoH, an efficient parallel training framework for MLLM training on heterogeneous

* Corresponding author

(a) (b) (c)

Fig. 1: Background and motivation. (a) shows he structure of popular MLLMs. MLLMs use encoders to extract features, project them into text space via a projector, and feed them into the LLM backbone to generate responses. The encoder and the LLM can be frozen to utilize off-the-shelf checkpoints. (b) compares the varying tendency of computational overhead and memory consumption across model layers between LLMs and the evaluated MLLM-3B. NF and FB denote the no-freeze and freeze-both, respectively. (c) shows the iteration time for training MLLM-3B using different parallel strategy planners under two frozen scenarios. MMoH1 and MMoH2 represent the parallel strategies for the respective frozen scenarios identified by the MMoH parallel strategy planner.

clusters. We introduce MMoH parallel strategy planner to generate the optimal parallel strategy, accounting not only for structural differences between LLMs and MLLMs but also for the varying parameter training states. Identifying the optimal parallel strategy is challenging, which has been proven to be an NP-hard problem[?]. To address this problem, we propose MMoH parallel strategy planner. The MMoH planner first proposes a demand-driven device topology method that identifies feasible topologies by co-optimizing the structure of MLLMs and the capabilities of heterogeneous devices. It then introduces a hybrid-granularity abstraction tailored to MLLMs' modular design, using multi-level partitioning to balance complexity and load balance. Finally, the planner performs frozen-aware model partitioning, which accounts for dynamic parameter states to select the best parallel strategy under each topology. Through this three-step process, our planner identifies an efficient parallel strategy through balanced load distribution and explicit consideration of the frozen parameters.

To better handle the varying training states of parameters in MLLMs, we propose an early-termination scheduler. This component detects and skips redundant operations when frozen parameters are present. Typically, pipeline parallelism (PP) involves three operations: (1) forward pass, (2) backward pass, and (3) inter-stage communication. Our scheduler removes unnecessary backward computations and redundant communications introduced by frozen parameters, significantly improving training efficiency without compromising convergence.

In summary, the contributions of this paper are as follows:

- We construct an MMoH parallel strategy planner. This planner performs joint optimization over the MLLM's structural characteristics, the resource capacities of heterogeneous devices, and the varying parameter states to determine the optimal parallel strategy.
- We propose an early-termination scheduler tailored to the frozen technique in MLLMs. It further reduces iteration time by identifying and eliminating unnecessary computations and communications.
- We conduct experiments using MLLMs of different sizes on two differently configured heterogeneous clusters. Experimental results demonstrate that MMoH achieves performance improvements of up to 1.77× compared to prevalent training approaches.

II. Background and Motivation

A. Multimodal Large Language Models

As shown in Fig. ??, a typical MLLM comprises three core modules[? ? ? ?]: an encoder, a projector, and an LLM. The encoder processes non-textual modality data to extract patch-level grid features and generates a modality feature embedding. Subsequently, the projector projects this embedding into a text semantic space that the LLM can comprehend. Finally, the LLM integrates the projected embedding with the input text embedding to produce the output.

Frozen [?] is a common technique employed during MLLM training. This technique differs fundamentally from the pre-training of LLMs. The MLLM training is typically performed in multiple stages, during which different modules are selectively frozen as illustrated in Fig. ??. As a result of this distinctive training procedure, four typical training scenarios emerge: 1) no-freeze: neither the encoder nor the LLM backbone, along with the projector, are frozen; 2) freeze-enc.: only the encoder is frozen; 3) freeze-llm: only the LLM backbone is frozen; 4) freeze-both: both the encoder and LLM backbone are frozen.

B. Distributed Training on Heterogeneous Clusters

Due to the substantial computational and memory requirements of LLMs, distributed training has become a prevalent approach. To enhance its efficiency, parallel strategies such as data parallelism (DP), tensor parallelism (TP) and pipeline parallelism (PP), are proposed. DP replicates the model across devices, splits the input data into mini-batches, and feeds each model replica with distinct mini-batches. During training, the all-reduce communication is invoked to average gradients across devices. TP partitions the tensors in the model into non-overlapping tensor shards and distributes these shards across devices. However, this approach necessitates frequent collective communications (e.g., all-reduce and all-gather), resulting in a substantial demand for network bandwidth. PP splits the mini-batch into micro-batches, employs a model partitioning to allocate model layers across devices, and orchestrates the training process in a pipeline manner [? ?]. Compared to DP and TP, PP only requires peer-to-peer (p2p) communications between

Fig. 2: The overview of MMoH. MMoH comprises two key components: MMoH parallel strategy planner and early-termination scheduler. The MMoH planner takes the MLLM configuration and heterogeneous cluster specifications as input and determines the optimal parallel strategy. The scheduler leverages the characteristics of the frozen technique to further improve performance by reducing unnecessary computation and communication overhead.

adjacent pipeline stages, imposing lower demands on network bandwidth.

Heterogeneous clusters, comprising multiple types of devices, are widely used for distributed training [? ? ?]. Megatron [?], however, ignores these variations and uniformly partitions model layers across devices, leading to suboptimal training performance on heterogeneous clusters. To tackle this issue, Whale [?] assigns devices with larger memory capacities to earlier stages within a device group and partitions model layers into stages based on the computation performance of the assigned device. Espresso [?] introduces a compute-to-memory ratio (CMR) metric to order devices within each device group. It employs Whale's layer partitioning strategy while complying with memory constraints.

III. MMoH Design

A. Overview

Fig. ?? illustrates the overview of MMoH, which consists of a MMoH parallel strategy planner (Section ??) and an early-termination scheduler (Section ??).

The MMoH planner takes cluster and model configurations as input. It first uses a demand-driven device topology method to find hardware setups that match the MLLM's needs. It then applies a hybrid-granularity abstraction to partition different modules appropriately. Next, frozen-aware model partitioning treats this as an integer programming (IP) problem to find the optimal scheme. For training, MMoH uses pipeline parallelism (PP) within device groups and data (DP) or tensor parallelism (TP) across groups. Additionally, its scheduler efficiently supports training with frozen parameters by identifying their states across phases, thereby eliminating redundant storage, computation, and communication.

B. MMoH parallel strategy planner

Prior work on parallel training strategies fails to account for the hybrid-module structure of MLLMs, resulting in suboptimal performance. To improve efficiency, we propose the MMoH parallel strategy planner. Our planner first introduces a new metric to identify suitable device topologies for a given MLLM. It then uses a hybrid-granularity abstraction to reorganize diverse model layers into multi-level model chunks. Finally, a frozen-aware partitioning scheme is applied, which uses a cost model to select the optimal partition strategy.

Demand-driven device topology. Inspired by the efficiency of MMoH1 and MMoH2 (Fig. ??), an effective device topology must accommodate the varying computational and memory demands of different MLLM layers to ensure optimal use of heterogeneous resources. To achieve this, the MMoH planner first profiles the computational overhead and memory consumption of model layers. We profile each unique layer once on a representative device since their computational cost measured in floating point operations(FLOPs) and memory consumption are consistent across repeated layers and show negligible variation across different heterogeneous devices. The total number of layers of the MLLM is denoted by l, which represents the sum of layers across its three modules. Let L_i and M_i denote the FLOPs and memory consumption of the i-th model layer, respectively. Within each device group, we denote P_j and Q_j, for $0 \leq j \leq s-1$, as the computational capability and memory capacity of the j-th device for each possible heterogeneous device permutation, where s equals to the PP degree. We then partition the model layers into s pipeline stages in proportion to the computational capabilities of the devices. This process results in two lists that reflect the total FLOPs and memory usage for each stage under a compute-balanced partitioning scheme, and we denote R_j, and T_j as the total FLOPs and memory consumption of j-th stage, respectively. To quantify the matching degree between the computational and memory demands of each pipeline stage under this hypothetical load distribution and the capabilities supplied by the candidate device topology, we propose a metric, MD, which is computed as follows:

$$\text{demand} = \{x_1, x_2, \cdots, x_{s-1}\} = \{\frac{a_1}{a_0}, \frac{a_2}{a_1}, \cdots, \frac{a_{s-1}}{a_{s-2}}\}, \tag{1}$$

$$\text{supply} = \{y_1, y_2, \cdots, y_{s-1}\} = \{\frac{b_1}{b_0}, \frac{b_2}{b_1}, \cdots, \frac{b_{s-1}}{b_{s-2}}\}, \tag{2}$$

$$MD_{(\text{demand, supply})} = e^{-\frac{1}{s-1}\sum_{j=1}^{s-1}(x_j - y_j)^2}, \tag{3}$$

where a_j equals the ratio of R_j to T_j, b_j equals the ratio of P_j and Q_j. We rank all candidate device topologies in descending order based on their MD scores and retain only the top half as final candidates.

Hybrid-granularity abstraction. In most MLLMs, encoder transformer layers are smaller—with fewer parameters and shorter sequences—than those in the LLM backbone. As a result, each encoder layer consumes relatively little computation and memory, so distributing them across stages has minimal impact on iteration time. In contrast, LLM transformer layers are significantly larger and process longer sequences. Partitioning even a single LLM layer across devices can lead to severe load imbalance, increasing iteration time and risking out-of-memory (OOM) errors, particularly on less capable devices. Other components—such as embedding layers, projectors (e.g., Q-Former), and output layers—also vary widely in size and resource requirements, further complicating partitioning. Due to this structural diversity, ap-

(a) Sub-module granularity for the encoder when the merge factor is 2.

(b) Sub-layer granularity for the LLM backbone.

Fig. 3: The illustration of hybrid-granularity abstraction. For the encoder, two encoder transformer layers are merged into an encoder-layers chunk. For the LLM, a single transformer layer is split into an attention chunk and an FFN chunk without introducing any influence on the communication volume.

plying a uniform partitioning strategy to all transformer layers is ineffective: it is too fine-grained for encoders (introducing overhead) and too coarse for LLM layers (causing imbalance and OOM). It also fails to account for non-transformer modules, limiting its applicability to MLLMs.

To address this, the planner introduces a hybrid-granularity abstraction that reorganizes diverse model layers into individual chunks. As shown in Fig. ??(a), smaller encoder transformer layers are grouped into larger encoder-layer chunks using sub-module granularity. For larger LLM transformer layers, sub-layer granularity [?] is applied—each layer is split into an Attention chunk and an FFN chunk (Fig. ??(b)), improving load balancing and reducing OOM risk on limited-memory devices. The number of encoder layers merged per chunk is determined through empirical profiling, ensuring that each encoder chunk does not exceed the computational or memory cost of a single Attention or FFN chunk. Non-transformer layers (e.g., embeddings, projectors, output layers) are treated at layer-level granularity, each assigned to a separate chunk based on its specific requirements.

Frozen-aware model partitioning. Based on the chunk list and candidate device topologies, the planner employs a frozen-aware model partitioning algorithm to optimally distribute chunks across heterogeneous devices. During training, frozen parameters require no updates. As a result, gradient synchronization time depends on the freezing status of each pipeline stage rather than the total number of parameters. To address this, MMoH introduces a novel frozen-aware partitioning algorithm. This method incorporates these dynamics into a cost model to optimize model partitioning and minimize iteration time.

Assuming the length of the chunk list is N, and the number of devices within a group is S. We represent the partition scheme as a sequence $P = \{p_1, p_2, ..., p_S\}$, where p_i is the number of chunks for i-th pipeline stage. With a candidate device topology, we can attain forward time list $\mathrm{FT} = \{F_1, F_2, \cdots F_S\}$ and backward time list $\mathrm{BT} = \{B_1, B_2, \cdots B_S\}$ for any partition scheme, where F_i, B_i can be estimated by summing up the execution time of individual chunks with a negligible error [?]. Let K as the index of the master stage [?], and M as the number of micro-batches. To account for the impact of the frozen parameters, we augment our cost model with frozen-aware communication analysis. Specifically,

Fig. 4: The effectiveness of early-termination scheduler. We demonstrate a 4-stage pipeline with 4 micro-batches labeled $0 \sim 3$, and the first two pipeline stages are frozen pipeline stages. Variations in forward and backward execution times result from the distribution of computational overhead and the heterogeneous computational performance.

we estimate the iteration time as:

$$\begin{aligned} T^* = &\sum_{i=1}^{K-1}(F_i + B_i) + 2\sum_{i=K+1}^{S}(F_i + B_i) \\ &+ (M - (S-K))(F_K + B_K) + \mathrm{COMM}_{time}, \end{aligned} \tag{4}$$

where the communication time corresponds to the maximum non-overlappable synchronization overhead across all pipeline stages, which we formally define as:

$$\mathrm{COMM}_{time} = max_{i=1}^{S}(AR_i - \sum_{j=1}^{i-1} B_i), \tag{5}$$

where AR_i means the communication overhead of i-th pipeline stage, which can be calculated by the number of active parameters within the i-th pipeline stage, if all chunks within a pipeline stage are frozen, this stage incurs no gradient synchronization overhead.

Therefore, we can get an optimal partition scheme by minimizing the iteration time under the following constraints:

$$\arg\min_{P=\{p_1,p_2,...p_S\}} \quad T^* \tag{6}$$

$$\text{s.t.} \sum_{i=1}^{S} p_i = N \tag{7}$$

$$F_i + B_i \le F_K + B_K, \quad \forall i \ne k \tag{8}$$

$$C_i^m + (S - i + 1)C_i^a \le D_i^m. \tag{9}$$

We employ the high-performance mathematical solver CPLEX[?] to solve this IP problem with negligible computational overhead.

C. Early-termination scheduler

We observe that if all stages before a frozen stage are also frozen, the backward computation for its input can be skipped. This stage also no longer requires gradient signals from later stages, eliminating associated communication. Since both input and output tensors are unused in backward passes, their memory can be freed. These optimizations reduce computation, communication, and memory overhead without affecting model accuracy. However, existing pipeline schedulers overlook these opportunities, still treating MLLMs as fully trainable and using fixed scheduling patterns.

Fig. ??(a) shows the standard 1F1B schedule in a 4-stage pipeline with 4 micro-batches. Fig. ??(b) illustrates its behavior when the first two stages are frozen. Fig. ??(c) introduces our early-termination scheduler, which detects frozen stages (e.g., the first two) and skips gradient computation for their inputs. Although omitted

for clarity, the scheduler also eliminates output tensor sends from frozen stages and prevents gradient receives from downstream stages, removing redundant peer-to-peer (p2p) communication. Furthermore, input and output tensors in frozen stages are freed from memory, as they are not needed for backpropagation. Together, these optimizations significantly improve training efficiency without affecting accuracy.

In such frozen configurations, we modify the cost model as follows:

$$T^* = \sum_{i=1}^{D} F_i + \sum_{i=D+1}^{K-1} (F_i + B_i) + 2 \sum_{i=K+1}^{S} (F_i + B_i) + (M - (S - K))(F_K + B_K) + \text{COMM}_{time}, \quad (10)$$

where D denotes the number of consecutive frozen pipeline stages at the front of the pipeline, the definition of K is revised to represent the index of the non-frozen pipeline stage with the longest computation time. This adjustment motivates a model partitioning strategy where frozen pipeline stages are assigned relatively heavier computational workloads, thereby reducing the burden on non-frozen stages and improving overall training efficiency.

IV. Evaluation

A. Experimental setups

Experimental platform. Our experiments are conducted on two heterogeneous GPU clusters, as summarized in Table ??. Cluster-S has four NVIDIA nodes: one with RTX 3090s, two with RTX 2080 Tis, and one with RTX 2080s. Each node contains four GPUs, totaling 216 GB of GPU memory. All nodes use dual Intel Xeon 4310 CPUs (2.10 GHz), 128 GB of DDR4 RAM, and are connected via 100Gbps InfiniBand. Cluster-L consists of eight NVIDIA nodes: four with RTX 3090s, two with RTX 2080 Tis, and two with RTX 2080s. Its CPU, memory, and network setup matches Cluster-S. RTX 3090 nodes use PCIe 4.0 for GPU interconnectivity, while RTX 2080/Ti nodes use PCIe 3.0. Both clusters run Ubuntu 18.04 with CUDA 11.8, cuDNN 8.7.0, NCCL 2.19.3, and PyTorch 2.2.

TABLE I: Configuration of heterogeneous clusters

Type (# of GPUs)	Index	GPU (# per node)	Total memory size (GB)
Cluster-S (16)	1	3090 (4)	216
	2,3	2080 Ti (4)	
	4	2080 (4)	
Cluster-L (32)	1,2,3,4	3090 (4)	536
	5,6	2080 Ti (4)	
	7,8	2080 (4)	

Models and datasets. We constructed two MLLMs by integrating distinct modules for evaluation, referred to as MLLM-3B and MLLM-12B. Detailed configurations of all modules are presented in Table ??. To align model memory demands with the clusters, MLLM-3B is trained on Cluster-S, while MLLM-12B is trained on Cluster-L. The training dataset is the LLaVA Visual Instruct dataset.

TABLE II: Configuration of Modules

MLLM	Module	Type	#Params	#Layers
MLLM-3B	ViT	Encoder	0.2B	24
	MLP-S	Projector	73M	2
	Mistral	LLM Backbone	2.8B	12
MLLM-12B	InternViT	Encoder	6B	45
	MLP-L	Projector	149M	2
	Yi	LLM Backbone	6B	20

(a) Training MLLM-3B on Cluster-S

(b) Training MLLM-12B on Cluster-L

Fig. 5: End-to-end throughput comparison across 4 frozen scenarios for training two different MLLMs on two heterogeneous clusters. Longer bars indicate higher throughput. OOM denotes an out-of-memory error.

Baselines and metrics. We evaluate MMoH against three state-of-the-art approaches: (1) Megatron [?] is a mainstream homogeneous training framework that has been adapted to support MLLM structures. (2) Whale [?] is an efficient training framework designed for large-scale models on heterogeneous GPU clusters. It organizes heterogeneous devices in descending order of memory capacity to optimize resource utilization. (3) Espresso[?] is a novel heterogeneous training framework that introduces a resource-aware stage placement strategy. It proposes a compute-memory ratio (CMR) metric. Based on this metric, GPUs are organized in ascending CMR order to jointly optimize computational throughput and communication efficiency during distributed training. Since MMoH preserves model accuracy, we focus our evaluation on training throughput (samples/second). For each experiment, we execute 10 warm-up iterations followed by 50 timed training iterations, and report the average throughput achieved during the measured phase.

B. End-to-End Training Performance

We compare the training throughput of MMoH against three baselines using MLLM-3B and MLLM-12B under four freezing setups: (1) no-freeze, (2) freeze-enc, (3) freeze-llm, and (4) freeze-both (details in Section ??). Each scenario changes memory usage and influences parallelization. We start with DP=4 and TP=1. If OOM occurs, we increment TP and decrease DP. If OOM persists when TP=4 (the maximum per-node parallelism), the configuration is deemed invalid. Results are shown in Fig. ??.

MLLM-3B. When training MLLM-3B on Cluster-S, Megatron shows the lowest throughput in all frozen scenarios. This is primarily because it was designed for homogeneous clusters and fails to balance work-

loads effectively across diverse GPUs, resulting in poor utilization. Although Megatron adopts Whale's device topology to prevent OOM errors in heterogeneous settings, it still suffers from significant load imbalance. Whale enhances performance by incorporating a load-balancing strategy across GPU types. Espresso further improves upon this by jointly optimizing load balance and communication overhead. MMoH builds on these advances with a demand-driven device topology that adapts to the computational and memory requirements of MLLMs, and a hybrid-granularity partitioning method tailored to each module's characteristics. Under the no-freeze scenario, MMoH delivers the highest throughput. While raising TP to 4 enables Megatron to train, frequent TP communication severely limits its performance. With the encoder frozen, MMoH achieves a 1.15× gain over Espresso; with the LLM backbone frozen, it improves upon Whale by 1.26×. In the freeze-both setting, MMoH maintains an advantage of 1.08×–1.72× over all baselines.

MLLM-12B. We evaluate MLLM-12B on Cluster-L. Under both the no-freeze and freeze-llm settings, Megatron fails to train due to OOM errors caused by load imbalance from its coarse-grained partitioning. Even after adjusting TP and DP, the first-stage devices remain overloaded—even when assigned the highest-memory devices using Whale's strategy. Whale avoids OOM by distributing the encoder across multiple stages instead of concentrating it in the first, enabling training in all scenarios. Espresso further improves throughput by co-optimizing load balance and communication. MMoH achieves the highest throughput in all cases, with particularly strong gains when the encoder or both encoder and LLM are frozen, thanks to its early-termination scheduler. Under the no-freeze scenario, both Espresso and MMoH significantly outperform Whale. MMoH achieves a 1.04× reduction in iteration time compared to Espresso, and over 1.26× improvement compared to Whale. When the LLM backbone is frozen, MMoH achieves a 1.26× and 1.11× improvement over Whale and Espresso, respectively. Under the freeze-enc. and freeze-both scenarios, MMoH demonstrates up to 1.77×, 1.28×, and 1.19× higher throughput than Megatron, Whale, and Espresso, respectively.

As shown in Table ??, We compare the model partitioning strategies of Whale, Espresso, and MMoH on two MLLMs, excluding Megatron's naive partitioning strategy. Whale and Espresso use single-layer granularity and are extended to support MLLMs. In contrast, MMoH adopts a hybrid-granularity approach for more balanced and efficient partitioning. For MLLM-3B, all three encoder layers form an encoder-layers chunk, while each LLM layer is split into two chunks. The input embedding, output layer, and projector are each treated as independent chunks. For MLLM-12B, each encoder layer forms a separate chunk due to its size, with other components handled similarly to MLLM-3B. As a result, MLLM-3B and MLLM-12B are reorganized into 35 and 88 chunks by MMoH, respectively, explaining the differences in total partitions compared to Whale and Espresso.

Fig. 6: The load balance analysis of MMoH and two heterogeneous approaches when training MLLM-12B under two frozen settings. The distribution is more concentrated representing the partition result is more balanced. The lower height indicates a shorter iteration time.

Model partitioning strategies comparison. To evaluate the impact of model partitioning on runtime, we compare average per-iteration computing times for Whale, Espresso, and MMoH. As shown in Fig. ??, under the no-freeze setting, Whale's allocation of high-performance GPUs to early stages misaligns with peak workloads, causing elevated overhead. Espresso and MMoH instead assign GPUs with higher memory capacity to later stages, better matching layer demands. Moreover, MMoH applies finer-grained partitioning in stages 5–7, smoothing workload spikes and reducing iteration time relative to Espresso. In the freeze-llm scenario, Whale requires TP=4 to avoid OOM, whereas Espresso and MMoH succeed with TP=2. Espresso places the 2080 Ti in stage 1, but MMoH opts for the lower-capacity 2080. This subtle adjustment allows MMoH to include four additional encoder layers early, balancing load and lowering computation peaks in later pipeline stages.

C. Ablation Study of Early-termination tcheduler

The effectiveness of the proposed early-termination scheduler is evaluated on MLLM-12B, with results demonstrated in Fig. ??. Due to the substantially lower performance of Megatron and Whale, our comparison is limited to Espresso and MMoH. In our evaluation, we focus on two frozen training scenarios: (1) freeze-enc. and (2) freeze-both, for which the early-termination scheduler is specifically designed. To isolate the impact of the scheduler, we evaluated four configurations: (1) the original Espresso; (2) Espresso integrated with the proposed scheduler (Espresso+); (3) MMoH without the scheduler (MMoH-); and (4) full MMoH with the scheduler enabled. As shown in Fig. ??(a), under freeze-enc., the original Espresso exhibits the lowest throughput. MMoH- achieves a 6.4% speedup over Espresso, while Espresso+ attains an 11.3% improvement. Full MMoH leads with a 19.2% gain over Espresso and 11.4% over MMoH-. Under freeze-both, only the original Espresso requires TP=4 while others train with TP=2. Full MMoH outperforms MMoH- and Espresso+ by 8.3% and 10.0%, respectively, and reduces iteration time by 19.0% relative to Espresso. We further analyze computation overhead under the freeze-enc. setting. As shown in Fig. ??(b), MMoH reallocates more computations to the first four

TABLE III: Model partitioning comparison across different frozen scenarios. A, B, and C denote NVIDIA RTX 3090, 2080 Ti, and 2080 GPUs.

Frozen Pattern	Whale (layer-granularity) MLLM-3B	Whale (layer-granularity) MLLM-12B	Espresso (layer-granularity) MLLM-3B	Espresso (layer-granularity) MLLM-12B	MMoH (hybrid-granularity) MLLM-3B	MMoH (hybrid-granularity) MLLM-12B
no freeze	[A,B,B,C] [30,3,3,2] 1.00x	[A,A,A,A,B,B,C,C] [15,15,17,9,3,3,3,2] 1.00x	[B,B,C,A] [26,3,2,7] 1.07x	[B,B,C,C,A,A,A,A] [7,7,6,6,15,11,8,7] 1.22x	[B,A,B,C] [11,14,6,4] 1.22x	[B,B,C,C,A,A,A,A] [7,7,6,6,18,16,13,15] 1.26x
freeze-enc.	[A,B,B,C] [31,3,3,1] 1.00x	[A,A,A,A,B,B,C,C] [38,11,5,5,2,3,2,1] 1.00x	[B,B,C,A] [28,3,2,5] 1.05x	[B,B,C,C,A,A,A,A] [8,8,6,6,22,13,13,12] 1.04x	[C,A,B,B] [12,13,6,4] 1.20x	[C,C,B,B,A,A,A,A] [12,11,11,11,11,11,11,10] 1.23x
freeze-llm	[A,B,B,C] [27,4,4,3] 1.00x	[A,A,A,A,B,B,C,C] [13,13,13,13,4,4,4,3] 1.00x	[B,B,C,A] [14,14,3,7] 1.15x	[B,B,C,C,A,A,A,A] [3,3,3,3,13,13,17,12] 1.20x	[B,B,A,C] [7,8,16,4] 1.26x	[C,C,B,B,A,A,A,A] [3,3,5,5,13,14,23,22] 1.33x
freeze-both	[A,B,B,C] [30,3,3,2] 1.00x	[A,A,A,A,B,B,C,C] [38,9,5,5,3,3,2,2] 1.00x	[B,B,C,A] [28,2,2,6] 1.08x	[B,B,C,C,A,A,A,A] [18,5,4,4,16,7,7,6] 1.20x	[B,A,B,C] [14,14,4,3] 1.18x	[A,B,B,C,C,A,A,A] [36,4,5,3,4,14,12,10] 1.28x

(a) End-to-end performance (b) Comp. overhead analysis

Fig. 7: Effectiveness of the early-termination scheduler. We integrate our scheduler into Espresso, denoted as Espresso+, and remove it from MMoH, denoted as MMoH–.

frozen stages, whose backward computations are skipped. This redistribution reduces the burden on later pipeline stages, enabling more efficient execution and shortening overall iteration time.

V. Conclusion

In this paper, we propose MMoH, an efficient framework for training multimodal large language models (MLLMs) on heterogeneous GPU clusters. It comprises an MMoH parallel strategy planner and an early-termination scheduler. The planner takes into account the modular nature of MLLMs, hardware heterogeneity, and the frozen technique to design an optimal parallel strategy. It begins by selecting suitable device topologies based on the resource demands of each module, then applies a hybrid-granularity abstraction to segment the model. Using this segmentation, it partitions the model into pipeline stages and maps them to the appropriate devices. During training, the scheduler further improves efficiency by removing redundant computation and communication made possible by frozen modules. Experiments show that MMoH achieves up to 1.77× speedup over existing training approaches.

Acknowledgment

This work is sponsored in part by the National Natural Science Foundation of China under Grant No. 62025208 and 62421002.